1. Chained-S3 on N83, m = 8, 9: parallel back enumeration

Run `n83_chain_sat_parallel_20261101`, 2026-11-01 20:46:09Z – 21:20:29Z. Protocol `PROTOCOL.md` frozen at `3045ec4d33` before the run (user: "now parallelize the back enumeration for m=8 and m=9"). Driver `examples/n83_chain_sat_bench.rs`; native analysis `examples/n83_chain_sat_analyze.rs` → `ANALYSIS.json`, `table.tsv`; graph by `plot.py` from `ANALYSIS.json` only. Host: one 4-vCPU x86-64 cloud VM, L0; m = 8 and m = 9 ran one after the other, each with the host to itself. Follow-up to 3216, where the single-threaded back enumeration ($C(170, 4) \approx 3.4 \cdot 10^7$ additions and lookups per query) was the whole per-query time.

Curve. `icv1-f2m83-t6151469093347-cdcc5432`; base: Frobenius union of span(1, z), 167 points. Seed 20261031: 3 public $[a]G$ and 5 planted sums per m (the targets of 3216). Per query 300,000 conflicts, 120 s.

1.1. What changed (`ChainS3Tables::parallel`)

`chain_s3_back_hits` factors out the per-query back enumeration (every signed $(m - d)$-multiset s, keeping $R + s$ when its x-fingerprint is in the depth-d set). With `parallel` it is split on the multiset's first (smallest) index across the rayon pool, each worker owning its scratch, path and hit list; the parts are concatenated in index order, so the hits equal the sequential ones in the same order (unit test `n83_parallel_back_hits_match_sequential`). The work is unchanged; only wall time can move. Lookup sets are built before the first query, single-threaded (not timed).

1.2. Results (measured, wall seconds)

m	d	verdicts, both arms	median s, A6 (1 thread) ⟶ A7 (4 threads): public	planted
8	4	3/3 refuted, 5/5 verified	41.7 ⟶ 11.4	41.8 ⟶ 11.7
9	5	3/3 refuted, 5/5 verified	57.0 ⟶ 13.7	55.2 ⟶ 13.9

1. **Soundness.** 32 records, 0 soundness flags; the two arms give the same verdict on every target; every witness re-adds to its target; no solver call on any target.
2. **Wall time.** Every A7 query (10.6–14.0 s) is faster than every A6 query on the same m (40.5–58.3 s): a 3.6–4.2× wall-clock reduction in the medians (41.7/11.4, 41.8/11.7, 57.0/13.7, 55.2/13.9) on 4 threads. Improvement under the frozen rule (lower median wall time, both arms deciding every query) at m = 8 and m = 9.
3. Per-query wall time at m = 9 is now ≈ 14 s; the one-off depth-5 build (single-threaded, 372,406,144 fingerprints) dominates a short run.

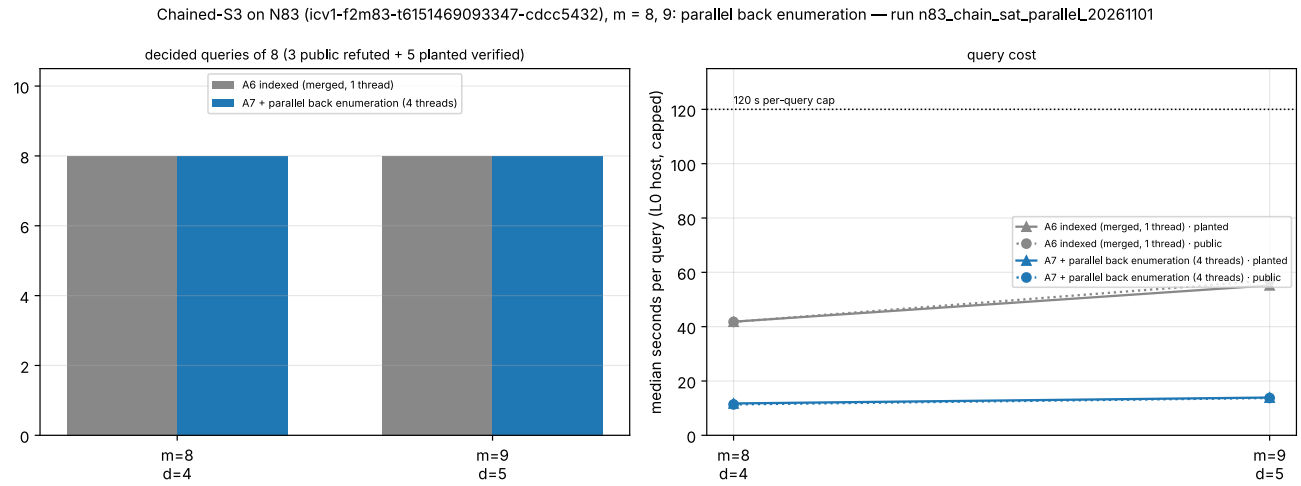


Figure 1: Decided queries (of 8) and median wall seconds per query, per arm.

1.3. What this is and is not

- A wall-clock speed-up on 4 vCPUs, not less work: the same $|B|^{m-d}$ enumeration, spread over threads. No CPU-time accounting was recorded.
- For m ≥ 5 the path is a meet-in-the-middle decomposition over the factor base; the SAT solver is not called on these targets. Decomposition stage only: no relation collection, no DLP, no leaderboard or `benchmarks/` row (validation gate). Matched rho on this curve: 1.51 s mean online (`research/n83_msweep_20261024`).

1.4. Limits

L0 shared cloud VM, 8 targets per m; wall time only. EKS (AGENTS.md §9a) not used: not an ecbench method, no aws CLI. Next lever (not measured): parallelize the one-off depth-d build, which is single-threaded (1,023 s at d = 5 in this run).